🔧 Fundamentals of Exception Handling in C++

Keywords: try, catch, throw

> **Definition:** *Exception handling* in C++ is a mechanism that separates error-handling code from regular code. It lets a program detect abnormal conditions, transfer control to a handler, and optionally recover.

- **try block** – contains code that may generate an exception.
- **throw statement** – creates an exception object/value and transfers control.
- **catch block(s)** – one or more handlers that match the thrown type.

General syntax

```
try {
    // code that may throw
}
catch (type1 var) {
    // handle type1
}
catch (type2 var) {
    // handle type2
}
// …
catch (typeN var) {
    // handle typeN
}
```

If no matching catch is found, std::terminate() is invoked (see later).

🚀 Throwing from Outside the try Block

An exception can originate in a function called **inside** a try block, even though the throw statement itself lives outside the visible try.

```
void Xtest(int test) {
    std::cout << "Inside Xtest, test is: " << test << "\n";
    if (test) throw test;          // throw from within the function
```

```
}

int main() {
    std::cout << "Start\n";
    try {
        std::cout << "Inside try block\n";
        Xtest(0);
        Xtest(1);                  // throws 1
        Xtest(2);                  // not reached
    }
    catch (int i) {
        std::cout << "Caught an exception -- value is: " << i << "\n";
    }
    std::cout << "End";
}
```

Output

```
Start
Inside try block
Inside Xtest, test is: 0
Inside Xtest, test is: 1
Caught an exception -- value is: 1
End
```

🏠 Localized try/catch Inside a Function

A try block can be placed *inside* a function, making the handling scoped to each call.

```
void Xhandler(int test) {
    try {
        if (test) throw test;
    }
    catch (int i) {
        std::cout << "Caught Exception #: " << i << '\n';
    }
}
```

Calling the function repeatedly creates independent handling for each invocation:

```
Start
Caught Exception #: 1
Caught Exception #: 2
Caught Exception #: 3
End
```

👤 User-Defined Exception Classes

Exceptions may be any type, including custom classes that carry rich information.

```
class MyException {
public:
    char msg[80];
    int   code;
    MyException() { *msg = 0; code = 0; }
    MyException(const char* s, int e) { strcpy(msg, s); code = e; }
};

int main() {
    int i;
    std::cout << "Enter a positive number: ";
    std::cin >> i;
    if (i < 0) throw MyException("Not Positive", i);
}
catch (MyException e) {
    std::cout << e.msg << ": " << e.code << "\n";
}
```

Output (input = -4)

```
Enter a positive number: -4
Not Positive: -4
```

The object transports both a message and an error code to the handler.

📚 Multiple catch Statements

A single try can be followed by several catch clauses, each handling a different type.

```
void Xhandler(int test) {
    try {
        if (test) throw test;            // int
        else throw "Value is zero";      // const char*
    }
    catch (int i) {
        std::cout << "Caught Exception #: " << i << '\n';
    }
    catch (const char* str) {
        std::cout << "Caught a string: " << str << '\n';
    }
}
```

Execution order

- catch(int) handles non-zero test.
- catch(const char*) handles the zero case.

🧬 Handling Derived-Class Exceptions

Because a base-class catch matches derived objects, the **most specific** handler must appear first.

```
class B { };
class D : public B { };

int main() {
    D derived;
    try { throw derived; }
    catch (D d) { std::cout << "Caught a derived class.\n"; }
    catch (B b) { std::cout << "Caught a base class.\n"; }
}
```

Output

```
Caught a derived class.
```

Placing catch(B) first would swallow the derived exception, preventing the specialized handler from running.

🎯 Catch-All Handler (catch(...))

The ellipsis matches *any* exception type, useful as a final safety net.

```
void Xhandler(int test) {
    try {
        if (test == 0) throw test;      // int
        if (test == 1) throw 'a';       // char
        if (test == 2) throw 3.14;      // double
    }
    catch (...) {
        std::cout << "Caught One!\n";
    }
}
```

All three calls produce the same message, guaranteeing that no exception escapes uncaught.

📏 Exception Specification (Throw List)

A function can restrict the types it may let propagate outward:

```
void Xhandler(int test) throw(int, char, double) {
    if (test == 0) throw test;          // int
    if (test == 1) throw 'a';           // char
    if (test == 2) throw 3.14;          // double
}
```

- Only int, char, or double may leave the function.

- Throwing any other type triggers std::unexpected(), which by default calls std::terminate().

> **Note:** Modern C++ (C++17 onward) deprecates dynamic exception specifications; noexcept is preferred.

🔁 Rethrowing Exceptions

Inside a catch, the statement throw; (no operand) re-propagates the current exception to an outer handler.

```
void Xhandler() {
    try {
        throw "hello";
    }
    catch (const char*) {
        std::cout << "Caught char * inside Xhandler\n";
        throw;                          // rethrow
    }
}
int main() {
    try { Xhandler(); }
    catch (const char*) {
        std::cout << "Caught char * inside main\n";
    }
}
```

Output

```
Caught char * inside Xhandler
Caught char * inside main
```

⚠️ terminate() and unexpected()

Situation	Function invoked

No matching catch	std::terminate()
Rethrow with no active exception	std::terminate()
Destructor throws during stack unwinding	std::terminate()
Function throws outside its throw-list	std::unexpected()

Both default to calling abort(), ending the program abruptly.

🛠️ Replacing the Default Handlers

Custom handlers can be installed with std::set_terminate and std::set_unexpected.

```
#include <exception>
#include <cstdlib>
#include <iostream>

void myTerminate() {
    std::cout << "Inside new terminate handler\n";
    std::abort();        // must not return
}

int main() {
    std::set_terminate(myTerminate);
    try {
        std::cout << "Inside try block\n";
        throw 100;                 // no matching catch
    }
    catch (double) {}              // does not match
}
```

Output

```
Inside try block
Inside new terminate handler
```

The program ends via abort() after the custom message.

🕵️ std::uncaught_exception()

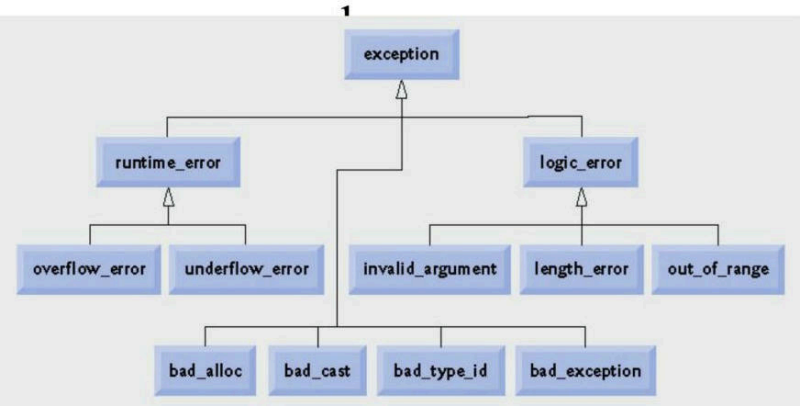
> **Definition:** Returns true while an exception is propagating and not yet caught.

Useful inside destructors to decide whether to perform cleanup that might itself throw.

```
struct Guard {
    ~Guard() {
        if (std::uncaught_exception())
            std::cout << "Leaving due to an exception\n";
        else
            std::cout << "Normal exit\n";
    }
};
```

📚 Standard Exception Hierarchy

The C++ Standard Library defines a base class std::exception and several derived types. The following flowchart (image) visualizes the hierarchy:



The diagram shows exception at the top, branching into runtime_error and logic_error, each with further specializations, plus direct links to bad_alloc, bad_cast, bad_typeid, and bad_exception.

- **std::exception** – base class; provides virtual const char* what() const noexcept.
- **Derived classes** (examples):
    - std::bad_alloc – memory-allocation failures (new).

- std::bad_cast – failed dynamic_cast.
- std::bad_typeid – typeid on a null polymorphic pointer.
- std::logic_error → std::invalid_argument, std::out_of_range, etc.
- std::runtime_error → std::overflow_error, std::underflow_error.

Typical usage

```
try {
    int* p = new int[100000000000];   // huge allocation
}
catch (const std::bad_alloc& e) {
    std::cout << "Exception caught: " << e.what() << '\n';
}
```

📊 Types of Exceptions in C++

Category	Typical Types	Example Throw
Built-in	int, char, double, const char*	throw 42;
User-defined class	Any class you define (e.g., MyException)	throw MyException("msg", code);
Standard library	std::exception derivatives (std::bad_alloc, std::out_of_range, ...)	throw std::out_of_range("index");
Catch-all	catch(...) matches everything	—
Derived-class ordering	Must catch most-derived first	catch(Derived) ... catch(Base) ...

⚡ Divide-by-Zero Example

```
void divide(double a, double b) {
    try {
        if (!b) throw b;                  // throw zero
```

```
        std::cout << "Result: " << a / b << '\n';
    }
    catch (double) {
        std::cout << "Can't divide by zero.\n";
    }
}
```

Sample Interaction

```
Enter numerator (0 to stop): 10
Enter denominator: 2
Result: 5
Enter numerator (0 to stop): 5
Enter denominator: 0
Can't divide by zero.
Enter numerator (0 to stop): 0
Enter denominator: 4
Result: 0
```

🆚 Error vs. Exception (Comparison)

> **Error** – a serious, usually unrecoverable problem (e.g., segmentation fault).
> **Exception** – a recoverable runtime condition that can be handled with try/catch.

Aspect	Error	Exception
When it occurs	Compile-time or runtime (system)	Runtime only
Severity	Often fatal (program abort)	Recoverable
Handling	OS/compiler, not via C++ constructs	try/catch/throw
Typical example	int x = 10 / 0; (crash)	if (den==0) throw "divide by zero";
Recoverability	No (program stops)	Yes (program can continue)